

SECTION 01

সহজ ভাষায় পরিচিতি (Intuition)

এই Concept কী এবং কেন দরকার?

Self-Attention হলো Transformer-এর হৃদয়। কিন্তু সাধারণ Dot-Product Attention-এ একটি বড় সমস্যা আছে: যখন vector-এর **dimension বড়** হয়, তখন dot product-এর মানগুলো অনেক বেশি ছড়িয়ে পড়ে — মানে **variance অনেক বেড়ে যায়**।

এই বেড়ে যাওয়া variance-এর কারণে softmax function এমন probability দেয় যে কিছু মান প্রায় ১০০% আর বাকিগুলো প্রায় ০% হয়ে যায়। ফলে training-এ vanishing gradient সমস্যা হয়। এটি ঠিক করার জন্যই $\sqrt{d_k}$ দিয়ে ভাগ করা হয় — এটিই **Scaled Dot-Product Attention**!

🏠 বাস্তব জীবনের উদাহরণ — ক্লাসরুম অ্যানালজি

❌ সমস্যার অবস্থা:

একটি ক্লাসরুমে বাচ্চাদের উচ্চতার পার্থক্য অনেক বেশি। কেউ খুব লম্বা, কেউ খুব বেঁটে। শিক্ষক বললেন "হাত তোলো" — শুধু লম্বারা দেখা যাচ্ছে। তাই শুধু লম্বাদের প্রশ্নই সমাধান হচ্ছে। বেঁটেরা ignore হয়ে যাচ্ছে — তাদের gradient শূন্য হয়ে যাচ্ছে!

✅ সমাধানের অবস্থা:

এবার সব বাচ্চার উচ্চতা প্রায় সমান। শিক্ষক হাত তুলতে বললে সবাই দেখা যায়। সবার প্রশ্ন সমান গুরুত্বে সমাধান হচ্ছে। সবার gradient সঠিকভাবে কাজ করছে — training স্থিতিশীল!

এখানে "উচ্চতার পার্থক্য" = dot product values-এর variance। "সমান করা" = $\sqrt{d_k}$ দিয়ে ভাগ করে variance নিয়ন্ত্রণ করা।



এক লাইনে সমস্যা ও সমাধান: উচ্চ-মাত্রার (high-dimensional) vector-এর dot product-এ variance বেশি হয়, তাই softmax বিকৃত হয়, training নষ্ট হয়। সমাধান: সব মানকে $\sqrt{d_k}$ দিয়ে ভাগ করে variance নিয়ন্ত্রণ করো।

SECTION 02

বিস্তারিত ব্যাখ্যা — আগের ভিডিওর পুনরাবৃত্তি

Self-Attention এর পূর্ববর্তী ধাপগুলো (পূর্ববর্তী ভিডিও থেকে)

ধরো আমাদের কাছে একটি sentence আছে: "Money Bank Grows"

Self-Attention প্রয়োগ করতে হলে নিচের প্রক্রিয়া অনুসরণ করতে হয়:

১

Embedding তৈরি করো: প্রতিটি শব্দের embedding তৈরি হয়। E_{money} (Money-র embedding), E_{bank} , E_{grows} — এগুলো প্রতিটি শব্দের সাংখ্যিক প্রতিনিধিত্ব।

২

তিনটি weight matrix নাও: W_Q , W_K , W_V । এগুলো trainable parameters — backpropagation-এ শেখে।

৩

Query, Key, Value vector তৈরি করো: প্রতিটি শব্দের embedding-কে W_Q , W_K , W_V দিয়ে গুণ করে Q_{money} , K_{money} , V_{money} পাওয়া যায়।

৪

Matrix গঠন করো: সব Query vector একসাথে স্তম্ভ করে Q matrix, সব Key vector স্তম্ভ করে K matrix, সব Value vector স্তম্ভ করে V matrix।

৫

Attention Score হিসাব করো: $Q \times K^T \rightarrow \text{Softmax} \rightarrow \times V \rightarrow \text{Contextual Embeddings}$ ।

সাধারণ SELF-ATTENTION FORMULA (আগের ভিডিও)

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T) \cdot V$$

⚠️ এবার নতুন ভিডিওর মূল পার্থক্য

"Attention is All You Need" পেপারটি দেখলে বোঝা যায়, সেখানকার formula-তে একটি অতিরিক্ত অংশ আছে। $Q \cdot K^T$ এর পরে $\sqrt{d_k}$ দিয়ে ভাগ করা হচ্ছে। এই কারণেই এটির নাম "Scaled" Dot-Product Attention।

SCALED DOT-PRODUCT ATTENTION (আসল পেপার)

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

এই ভিডিওর লক্ষ্য: d_k কী এবং কেন $\sqrt{d_k}$ দিয়ে ভাগ করতে হয় — এটা সম্পূর্ণভাবে বোঝানো।

SECTION 03

 d_k কী? — Key Dimension

d_k = Key Vector-এর Dimension

d_k হলো **dimension of the Key vector**। অর্থাৎ K_money, K_bank, K_grows — এই প্রতিটি Key vector-এ কতগুলো সংখ্যা আছে।

Key Vector	Dimension (d_k)	উদাহরণ
K_money	3	[0.2, 0.8, 0.5]
K_bank	3	[0.9, 0.1, 0.3]
K_grows	3	[0.4, 0.6, 0.7]

এখানে d_k = 3 । যদি embedding 512-dimensional হয়, তাহলে d_k হবে 512। উল্লেখ্য: সাধারণত $d_Q = d_K = d_V$ (সব সমান রাখা হয়)।

Key Vector-এর Dimension কীভাবে নির্ধারিত হয়?

ধরো embedding 3-dimensional। তাহলে W_K matrix হবে 3×3 । Embedding (1×3) কে W_K (3×3) দিয়ে গুণ করলে আউটপুট হবে (1×3) । তাই Key vector-ও 3-dimensional $\rightarrow d_k = 3$ ।



যদি W_K matrix-এর shape ভিন্ন হয়, তাহলে d_k , d_Q , d_V ভিন্ন হতে পারে। কিন্তু সরলীকরণের জন্য সাধারণত সবাইকে সমান ধরা হয়।

SECTION 04



Dot Product-এর Nature — মূল সমস্যার উৎস

Variance ও SD (Standard Deviation) কী?

সামনের আলোচনা বোঝার আগে পরিসংখ্যানের এই দুটি বেসিক জিনিস ক্লিয়ার থাকা জরুরি:

- **SD (Standard Deviation):** এর মানে হলো সংখ্যাগুলো তাদের মূল গড় (Mean) থেকে গড়ে ঠিক কতটা দূরে ছড়িয়ে আছে। এটি মানুষের সহজে বোঝার মাপকাঠি।
- **Variance (ভেরিয়েন্স):** এটি হলো SD-এর বর্গ (Square)। এটি গাণিতিক হিসাব-নিকাশে (যেমন ট্রান্সফর্মারে) ব্যবহার করা হয়, কারণ গাণিতিক নিয়মে স্বাধীন জিনিসের ভেরিয়েন্স সরাসরি যোগ করা যায়, যা SD-এর ক্ষেত্রে করা যায় না। ভেরিয়েন্স বোঝায় সংখ্যাগুলো নিজেদের মধ্যে কতটা "অস্থির" বা "বিস্ফুট"।

$Q \times K^T$ Matrix-এর ভেতরে কী লুকিয়ে আছে?

Q matrix তিনটি query vector দিয়ে তৈরি, K^T তিনটি key vector দিয়ে। তাই $Q \times K^T$ করলে মোট **৩টি ভিন্ন ভিন্ন vector-vector dot product** হিসাব হয়। প্রতিটি dot product একটি scalar সংখ্যা।

এই ৩টি সংখ্যার জন্য আমরা **mean** ও **variance** হিসাব করতে পারি। এবার মূল পর্যবেক্ষণ:

Dot Product-এর Nature (মূল আবিষ্কার)

Low-Dimensional Vectors:

ছোট dimension-এর vector-দের dot product করলে যে সংখ্যাগুলো পাওয়া যায়, সেগুলোর **variance কম থাকে**।

High-Dimensional Vectors:

বড় dimension-এর vector-দের dot product করলে যে সংখ্যাগুলো পাওয়া যায়, সেগুলোর **variance অনেক বেশি হয়**।



ডাইমেনশন ছোট-বড় কীভাবে হয়? (Confusion Clear):

একটি নির্দিষ্ট ট্রান্সফর্মার মডেলের ভেতর (যেমন আপনার তৈরি করা কোনো RAG মডেলে) রানটাইমে ডাইমেনশন (q, k, v এর shape) একই থাকে, এটি ছুট করে পরিবর্তন হয় না। এখানে "ছোট-বড়" বলতে **বিভিন্ন মডেলের আর্কিটেকচারাল ডিজাইনের পার্থক্য** বোঝানো হচ্ছে। যেমন, একটি ছোট মডেলে ডাইমেনশন হতে পারে মাত্র 3 বা 64, কিন্তু GPT-3 এর মতো বিশাল মডেলে ডাইমেনশন হতে পারে 12288। যখন আমরা বড় ডাইমেনশনের মডেল বানাতে যাই, তখনই গাণিতিকভাবে এই Variance বিস্তারণের সমস্যাটি দেখা দেয়।

HISTOGRAM: বিভিন্ন DIMENSION-এর 1000 DOT PRODUCTS-এর বিতরণ

3D vectors	spread: -5 থেকে +5
100D vectors	spread: -10 থেকে +10
1000D vectors	spread: -30 থেকে +30 ⚠️

Nitish-এর কোড পরীক্ষায় প্রমাণিত: 1000টি pair-এর dot product নিয়ে histogram আঁকলে dimension বাড়ার সাথে X-axis-এ spread (variance) বাড়তে থাকে।

SECTION 05



High Variance কেন বড় সমস্যা?

Softmax-এর Nature বোঝা

Softmax একটি সেট সংখ্যাকে probability-তে রূপান্তরিত করে, যেখানে সবার যোগফল = ১। কিন্তু Softmax-এর exponential প্রকৃতির কারণে, বড় সংখ্যাগুলো সব প্রবাবিলিটি টেনে নেয়:

Low Variance Input (Scaled)

Input: [8, 0.8, -9]

Output: [80%, 15%, 5%]

→ সফটম্যাক্স মানগুলোকে আর Extreme মানে নিয়ে যায় না। Training সবার উপর সমান মনোযোগ দেয়।

High Variance Input (Unscaled)

Input: [100, 10, -120]

Output: [~100%, ~0%, ~0%]

→ সবচেয়ে বড় মানটাকে প্রায় ১০০% করে দেয় এবং বাকিগুলোকে একেবারে ০% করে দেয়। বাকিরা ignore হয়!

Vanishing Gradient সমস্যা — কীভাবে হয়?

- ১ High-dimensional vectors → High variance dot products (কিছু খুব বড়, কিছু খুব ছোট)
- ২ Softmax লাগালে → Extreme probabilities (কিছু $\approx 100\%$, কিছু $\approx 0\%$)
- ৩ Backpropagation-এ সব focus যায় বড় probability-র উপর। ছোট probability-গুলোর gradient প্রায় শূন্য হয়ে যায়।
- ৪ **Vanishing Gradient!** ছোট probability-দের সংশ্লিষ্ট parameters update হয় না → Training process ব্যর্থ হয়।

🏠 ক্লাসরুম অ্যানালজি (বিস্তারিত)

সমস্যার ক্লাস: Softmax matrix-এ কিছু নম্বর বড় = কিছু বাচ্চা খুব লম্বা। Training শুধু বড় নম্বরগুলো ঠিক করতে মনোযোগ দেয় = Teacher শুধু লম্বাদের question শোনে। ছোট নম্বরগুলো (বেঁটে বাচ্চারা) ignore হয়, তাদের gradient শূন্য হয়, তারা train হয় না।

সমাধানের ক্লাস: $\sqrt{d_k}$ দিয়ে ভাগ করে variance কমাও = সব বাচ্চার উচ্চতা সমান করো। এখন softmax সব মানকে প্রায় সমান probability দেয় = Teacher সবার প্রশ্ন শোনে। সব parameters সমানভাবে update হয় = সবার learning হয়। Training stable!

SECTION 06



গাণিতিক প্রমাণ — কেন ঠিক $\sqrt{d_k}$?

Variance এবং Dimension-এর সম্পর্ক প্রমাণ করা

আমরা শুধু প্রথম row-তে ফোকাস করি (পুরো matrix-এর সমস্যা একই)। প্রথম row-তে আছে: $s_{11} = q_1 \cdot k_1$,
 $s_{12} = q_1 \cdot k_2$, $s_{13} = q_1 \cdot k_3$ ।

এই সংখ্যাগুলো random variable। আমরা তাদের **expected variance** বের করতে চাই।

Step-by-Step গাণিতিক যুক্তি (বিস্তারিত)

📌 1D Vector-এর ক্ষেত্রে:

ধরি $v_1 = a$, $v_4 = b$, $v_5 = c$, $v_6 = d$ । তাহলে: $s_{11} = a \times b$, $s_{12} = a \times c$, $s_{13} = a \times d$ । এই random variable-টিকে বলি X । এর variance = $\text{Var}(X)$ ।

📌 2D Vector-এর ক্ষেত্রে:

এখন প্রতিটি vector 2D। $v_1 = [a, b]$, $v_4 = [d, e]$ ।
এখন তোমার ভেক্টরে ২টি করে সংখ্যা আছে। ডট প্রোডাক্টের নিয়ম অনুযায়ী, তুমি সামনাসামনি থাকা সংখ্যাগুলো গুণ করে তারপর যোগ করবে: $s_{11} = (a \times d) + (b \times e)$ । এটি এখন দুটি পদের যোগফল। এই random variable-টিকে বলি Y । $\text{Var}(Y) \approx 2 \times \text{Var}(X)$ । কারণ দুটি স্বাধীন term যোগ হয়েছে (পরিসংখ্যানের নিয়ম অনুযায়ী ভেরিয়েন্স সরাসরি যোগ করা যায়)।

📌 3D Vector-এর ক্ষেত্রে:

Dot product এখন তিন পদের যোগফল। Random variable = Z । $\text{Var}(Z) \approx 3 \times \text{Var}(X)$ ।

সাধারণ নিয়ম (GENERAL RULE)

d-dimensional হলে: $\text{Var} = d \times \text{Var}(X)$

Dimension (d)	Variance	সমস্যা?
1	$1 \times \text{Var}(X)$	✅ স্বাভাবিক
2	$2 \times \text{Var}(X)$	⚠️ বাড়াচ্ছে
3	$3 \times \text{Var}(X)$	⚠️ বাড়াচ্ছে
512	$512 \times \text{Var}(X)$	❌ অনেক বেশি!
d_k	$d_k \times \text{Var}(X)$	❌ নিয়ন্ত্রণ দরকার

Variance নিয়ন্ত্রণের সহজ কৌশল

যদি আমার কাছে সংখ্যাগুলো থাকে: 10, 20, 30, 40, 50, 60, 70 । এদের variance = 400 (অনেক বেশি)। এখন সবাইকে 10 দিয়ে ভাগ করি: 1, 2, 3, 4, 5, 6, 7 । নতুন variance = 4। অনেক কম!



Mathematical Rule: যদি random variable X -এর variance = $\text{Var}(X)$, এবং নতুন variable $Y = X/c$, তাহলে $\text{Var}(Y) = \text{Var}(X) / c^2$ ।

কেন ঠিক $\sqrt{d_k}$ দিয়ে ভাগ করা হয়? (ভাগ করে প্রমাণ)

আমাদের লক্ষ্য: dimension যাই হোক, variance সবসময় $\text{Var}(X)$ থাকবে।

d=2

সমস্যা: $\text{Var} = 2 \times \text{Var}(X)$ ।

সমাধান: $\sqrt{2}$ দিয়ে ভাগ করো।

প্রমাণ: $\text{Var}(Y/\sqrt{2}) = \text{Var}(Y)/2 = 2 \times \text{Var}(X)/2 = \text{Var}(X)$ ✓

d=3

সমস্যা: $\text{Var} = 3 \times \text{Var}(X)$ ।

সমাধান: $\sqrt{3}$ দিয়ে ভাগ করো।

প্রমাণ: $\text{Var}(Z/\sqrt{3}) = \text{Var}(Z)/3 = 3 \times \text{Var}(X)/3 = \text{Var}(X)$ ✓

d=d_k

সমস্যা: $\text{Var} = d_k \times \text{Var}(X)$ ।

সমাধান: $\sqrt{d_k}$ দিয়ে ভাগ করো!

প্রমাণ: $\text{Var}(\text{result}/\sqrt{d_k}) = \text{Var}(\text{result})/d_k = d_k \times \text{Var}(X)/d_k = \text{Var}(X)$ ✓

চূড়ান্ত সিদ্ধান্ত

যে কোনো dimension-এ variance = $\text{Var}(X)$ রাখতে
→ প্রতিটি মানকে $\sqrt{d_k}$ দিয়ে ভাগ করো

Key vector-এর dimension = d_k । আমাদের উদাহরণে $d_k = 3$, তাই $\sqrt{3}$ দিয়ে ভাগ করা হবে। Formula দাঁড়ায়:

$\text{softmax}(QK^T / \sqrt{d_k}) \cdot V$ ।



সবকিছুর ভেরিয়েন্স ১ রাখলে মডেল শিখবে কীভাবে?

ভেরিয়েন্স ১ রাখার মানে এই নয় যে সব ভ্যালু সমান হয়ে যায় বা মডেল বোকা হয়ে যায়। এর মানে হলো সংখ্যাগুলোর "স্কেল" বা সাইজ লিমিটের ভেতরে রাখা। ইনপুট ভেক্টরগুলোর ভেতরের সূক্ষ্ম পার্থক্যগুলো (রিলেটিভ ডিফারেন্স) ঠিকই বজায় থাকে। ভেরিয়েন্স ১ থাকলে সফটম্যাক্স ওভাররিঅ্যাক্ট করে না, ফলে গ্রেডিয়েন্ট সুন্দরভাবে ফ্লো করে এবং মডেল ব্যাকপ্রোপাগেশনের মাধ্যমে ওই ওয়েটগুলোকেই (Weights) আপডেট করে নতুন জিনিস শিখতে পারে!

SECTION 06.5

Layer Normalization ও Variance ১ করার ম্যাড্রিক

মডেলের ভেতরে ভেরিয়েন্স ৬ কীভাবে জন্ম নেয়?

আমরা বলি যে বেসলাইন অবস্থায় (ডেট প্রোডাক্টের আগে) প্রতিটি জোড়ার গুণফলের ভেরিয়েন্স ৬ থাকে। কিন্তু এটি জাদুকরীভাবে হয় না! এর পেছনে দুটি কারণ আছে:

১. Weight Initialization:

সব ওয়েট ম্যাট্রিক্সের (W_Q , W_K , W_V) ভেরিয়েন্স শুরুতেই ৬ রাখার চেষ্টা করা হয়। Xavier Initialization বা He Initialization এর মাধ্যমে বিজ্ঞানীরা এমন র্যান্ডম সংখ্যা বসান যেন তাদের ভেরিয়েন্স ৬ থাকে।

২. Layer Normalization:

ট্রান্সফর্মার মডেলে LayerNorm নামক একটি দারোয়ান বা ফিল্টার বসানো থাকে। এর একমাত্র কাজ হলো যেকোনো উদ্ভট বা ছন্নছাড়া সংখ্যাকে ধরে এনে তার ভেরিয়েন্স জোর করে ৬ বানিয়ে দেওয়া!

উদাহরন: [০, ৫০, ৬০]-এর ভেরিয়েন্স কীভাবে ৬ হয়?

ধরো, মডেলের ভেতরে তোমার দেওয়া [0, 50, 60] ভেক্টরটি তৈরি হয়ে গেল। এর ভেরিয়েন্স অনেক বেশি (প্রায় ৬৮৮)। Layer Normalization এখানে ঠিক ৩টি ধাপে কাজ করবে:

১

গড় (Mean) বের করা: 0, 50 এবং 60 এর গড় হলো 36.67।

২

গড় বিয়োগ করা: প্রতিটি সংখ্যা থেকে এই গড় বিয়োগ করা হয়।

$[0 - 36.67, 50 - 36.67, 60 - 36.67] = [-36.67, 13.33, 23.33]$ (এখন এদের গড় শূন্য)।

৩

Standard Deviation (SD) দিয়ে ভাগ: এরপর সংখ্যাগুলোকে তাদের SD (যা এখানে প্রায় 26.25) দিয়ে ভাগ করা হয়।

$$-36.67 \div 26.25 = -1.39$$

$$13.33 \div 26.25 = 0.51$$

$$23.33 \div 26.25 = 0.89$$

নতুন ভেক্টর: $[-1.39, 0.51, 0.89]$

SD (২৬.২৫) কীভাবে এলো? খাতা-কলমের হিসাব:

- বর্গ (Square): $(-36.67)^2 = 1344.69$, $(13.33)^2 = 177.69$, $(23.33)^2 = 544.29$
- Variance (গড়): $(1344.69 + 177.69 + 544.29) / 3 = 688.89$
- SD (বর্গমূল): $\sqrt{688.89} \approx 26.25$



প্রমাণ (কেন ভেরিয়েন্স ১ হলো): আমরা জানি, কোনো মানকে একটি ধ্রুবক সংখ্যা দিয়ে ভাগ করলে, তার ভেরিয়েন্স সেই সংখ্যার বর্গ (Square) দিয়ে ভাগ হয়ে যায়। এখানে আমরা SD দিয়ে ভাগ করেছি, আর SD^2 মানেই ভেরিয়েন্স!

নতুন ভেরিয়েন্স = পুরোনো ভেরিয়েন্স / $SD^2 = 688.89 / 688.89 = 1$ ✓

SECTION 07



Visual Diagrams — সম্পূর্ণ চিত্র

Scaled Dot-Product Attention — সম্পূর্ণ প্রবাহ চিত্র

FLOW: "MONEY BANK GROWS" SENTENCE → CONTEXTUAL EMBEDDINGS



Variance Control — Before vs After Scaling

WITHOUT SCALING (d=512):

Dot products: [-180, +210, -5, +195, -170, +190, -8, +185, -175]

Variance: খুব বেশি! → Softmax biased হয়ে যায়

Softmax output: [≈0%, ≈99%, ≈0%, ≈98%, ≈0%, ≈97%, ≈0%, ≈96%, ≈0%]

⚠ Extreme! Training ব্যর্থ! Vanishing Gradient!

WITH SCALING ($\div\sqrt{512} \approx \div22.6$):

Scaled: [-8.0, +9.3, -0.2, +8.6, -7.5, +8.4, -0.4, +8.2, -7.7]

Variance: নিয়ন্ত্রিত! → Softmax ভারসাম্যপূর্ণ

Softmax output: [0.12, 0.18, 0.10, 0.16, 0.11, 0.15, 0.09, 0.14, 0.12]

✅ Balanced! Training stable! Good gradients!



Mathematical Rule — Variance of Scaled Variable

Mathematical Property (ব্যবহৃত নিয়ম):

যদি X একটি random variable এবং $\text{Var}(X)$ তার variance,
নতুন variable $Y = c \times X$ হলে:

$$\text{Var}(Y) = c^2 \times \text{Var}(X)$$

আমাদের প্রয়োগ:

d -dimensional vectors $\rightarrow$ Dot products-এর $\text{Var} = d \times \text{Var}(X)$

যদি প্রতিটি value কে $(1/\sqrt{d_k})$ দিয়ে গুণ করি:

$$c = 1/\sqrt{d_k}$$

$$\begin{aligned}\text{নতুন Var} &= c^2 \times (d_k \times \text{Var}(X)) \\ &= (1/d_k) \times d_k \times \text{Var}(X) \\ &= \text{Var}(X) \quad \checkmark \text{ সবসময় সমান!}\end{aligned}$$

তাই $\sqrt{d_k}$ দিয়ে ভাগ করলে, dimension যাই হোক,
 $\text{variance} = \text{Var}(X) \rightarrow \text{constant} \rightarrow \text{stable training!}$

SECTION 08



চূড়ান্ত সারসংক্ষেপ

Scaled Dot-Product Attention — সম্পূর্ণ Process

- ১ **Embedding তৈরি:** প্রতিটি শব্দের embedding নাও। W_Q , W_K , W_V দিয়ে গুণ করে Q , K , V matrix তৈরি করো।
- ২ **Score Matrix:** $Q \times K^T$ করো। এই matrix-এর মানগুলোতে high variance সমস্যা আছে।
- ৩ **Scaling Step (নতুন):** Score matrix-এর প্রতিটি মানকে $\sqrt{d_k}$ দিয়ে ভাগ করো। এতে variance নিয়ন্ত্রিত হয়।
- ৪ **Softmax:** Scaled score-এর উপর softmax লাগাও। এখন probability সুস্থ (balanced), extreme নয়।
- ৫ **Final Output:** Attention weights $\times V$ matrix। পাওয়া যায় contextual embeddings — আমাদের চূড়ান্ত লক্ষ্য।

চূড়ান্ত FORMULA — TRANSFORMERS-এ ব্যবহৃত

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

সমস্যার কারণ

High-dimensional vectors → High variance
dot products → Biased softmax →
Extreme probabilities → Vanishing
gradient → Training failure

সমাধান

$\sqrt{d_k}$ দিয়ে ভাগ → Variance নিয়ন্ত্রিত →
Balanced softmax → Stable gradients →
Successful training ✓



মূল সিদ্ধান্ত: অনেকে শুধু এক লাইনে বলেন "unstable gradients এড়াতে scaling করা হয়"। কিন্তু এই গাইডে আমরা দেখলাম কেন ঠিক $\sqrt{d_k}$ — কারণ dimension-এর সাথে variance linear হারে বাড়ে ($d \times \text{Var}(X)$), তাই $\sqrt{d_k}$ দিয়ে ভাগ করলে variance আবার $\text{Var}(X)$ -এ ফিরে আসে। এটি গণিতগতভাবে প্রমাণিত, অনুমানের উপর নির্ভরশীল নয়।

Scaled Dot-Product Attention — Technical Guide

Nitish-এর "100 Days of Deep Learning" সিরিজের উপর ভিত্তি করে তৈরি • সম্পূর্ণ বাংলায়

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$